

TD : SGF

Exercice 1

1. Soit un disque dur avec les caractéristiques suivantes:

- Composé de 6 plateaux
- chaque face d'un plateau contient 10 pistes
- chaque piste contient 15 secteurs
- sachant que chaque bloc physique égal à 3 Secteurs
- et chaque secteur égal 512 octets

Calculer la taille disque dure en terme de nombre de faces, nombre de cylindres, nombre de pistes, nombre de secteurs, nombre de blocs physiques et taille en Octets ?

faces (tetes)	cylindres	pistes	secteurs	blocs physiques	Octets
12	10	10*12	12*10*15= 1800	1800/3=600	1800*512

2. Considérons un SGF utilisant une méthode d'allocation chaînée indexée à base d'une table FAT. Chaque entrée de la table FAT a une taille de 24 bits. Pour un disque de 32 Go, quelle est la taille minimale d'allocation de fichier dans ce système ? Justifiez votre réponse.

Solution:

Un disque de 32 Go a 2^{35} octets de stockage, si chaque entrée de la FAT a 24 bits, il peut être tout au plus 2^{24} unités d'allocation (FAT par entrée), chaque unités d'allocation doit être de 2^{11} octets = 2KB

3. Peut-on mettre en place les liens symboliques dans le système de fichiers FAT? Pourquoi?

Solution:

Oui. Il peut être fait par l'attribution d'un bit dans le répertoire pour indiquer que cette entrée est un nom symbolique. Ensuite, le nom complet est stocké comme contenu du fichier qui sert de lien symbolique. Le système d'exploitation doit alors être modifié pour reconnaître qu'il s'agit d'un lien symbolique, et donc d'effectuer une recherche supplémentaire pour accéder au fichier.

Exercice 2:

Prenons l'exemple d'un système de fichier très simple pour un petit disque. Chaque secteur sur le disque tient sur deux entiers, et tous les blocs de données, blocs indirects et les i-noeuds sont de taille 1 secteur de disque (chacun contient 2 entiers). Tous les fichiers stockés sur le disque sont interprétés comme des répertoires par le système de fichiers (il n'y a pas de "fichiers de données"). Le système de fichiers définit les structures de données suivantes sur le disque:

i-noeud = 1 pointeur sur un bloc de données + 1 pointeur sur un bloc indirect

bloc indirect = 2 pointeurs vers les blocs de données

Un répertoire de fichiers contenant zéro ou plusieurs paires d'entiers, le premier entier de chaque paire est un nom de fichier et le second est l'i-noeud du fichier.

La valeur "99" signifie un pointeur nul lorsque l'on se réfère à un bloc disque adresse ou nom de répertoire.

Un répertoire vide a un bloc disque avec le contenu "99 99". L' i-noeud pour le répertoire racine est "/" est de 0.

Les données suivantes sont stockées sur le disque:

Table des I-nœuds :

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
10 6	7 99		8 99			3 99									

bloc disque:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	32 3		96 1			1 99	99 99	99 99		57 6					

- a. Combien d'entrées au maximum un répertoire peut contenir ? (Chaque entrée est une paire de nombres entiers)

Solution: 3 (un bloc de données et de deux autres blocs de données fait par un bloc indirect)

- b. Donner tous les répertoires stockés sur le disque (avec les chemins complets), ainsi que les noms des fichiers stockés dans chaque répertoire.

Solution:

répertoire le nom de chemin	i-noeud	blocs indirects	les blocs de données	contenu (sous-répertoires)
/	0	6	10 1	/32, /57
/32	3	n/a n / a	8	n/a
/57	6	n/a n / a	3	/57/96
/57/96	1	n/a n / a	7	n/a

- c. Modifier les structures de données ci-dessus afin d'ajouter un répertoire vide appelé "87" dans le répertoire "/"

Solution:

répertoire le nom de chemin	i-noeud	blocs indirects	les blocs de données	contenu (sous-répertoires)
/87	9	n/a n / a	14	n/a
/	0	6	10 1 13	/32, /57, /87

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
10 6	7 99		8 99			3 99			14 99						

Blocs disque:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
	32		96			1	99	99		57			87	99	
	3		1			13	99	99		6			9	99	

Exercice 3 :

Le système de fichiers « FastFile » utilise une table i-noeud pour organiser les fichiers sur le disque. Chaque i-noeud est constitué d'un identificateur d'utilisateur (2 octets), trois dates (4 octets chacun), les bits de protection (2 octets), le nombre de liens (2 octet), un type de fichier (2 octets) et la taille (4 octets). En outre, l'i-noeud contient 13 index directs, 1 index vers une table à un seul niveau (simple indirection), 1 index vers une table à un deux niveaux (double indirection), et 1 index vers une table à trois niveau (triple indirection). Le système de fichiers stocke aussi les premiers 436 octets de chaque fichier dans l'i-noeud.

1. Supposons un secteur de disque est de 512 octets, et supposons que toute table auxiliaire tiens sur un secteur, quelle est la taille maximale d'un fichier dans ce système.
2. Est-t-il un bénéfice pour les 436 premiers octets du fichier soit inclut dans l'i-noeud?

Solution:

1. Note: La première chose que vous devez déterminer la taille d'un index (2 octets ou 4 octets). Étant donné que nous avons un niveau 3-schéma d'indexation, vous pouvez rapidement calculer le nombre de secteurs que vous pouvez obtenir par index de 2 octets et des index de 4 octets. Vous verrez que les index à 2 octets ne fonctionne pas. Cela vous donne jusqu'à 256 indices par secteur, avec une taille maximale de $436 + 13 * 512 + 1 * 256 * 512 + 1 * 256 * 256 * 512 + 1 * 256 * 256 * 256 * 512 = n$. Le problème avec cette analyse est que vous avez beaucoup de secteurs de disque qu'on peut pas les adressés sur 2 octets. Vous pouvez utiliser 3 octets, mais peut devenir laid. Donc, nous allons utiliser un index à 4 octets, qui donnera 128 entrées par secteur, et la réponse correcte est: $436 + 13 * 512 + 1 * 128 * 512 + 1 * 128 * 128 * 512 + 1 * 128 * 128 * 128 * 512 = 1082203060$, environ 1G.

2. Oui. La plupart des fichiers sont de petite taille. Si la taille du fichier est 436 octets ou moins, alors l'ensemble des fichiers peuvent être lus et écrits dans une opération de disque sans avoir à faire un accès indépendant à l'i-noeud.

Exercice 4:

Peut-on mettre en place les liens physiques (durs) dans le système DOS? Pourquoi?

Solution:

Non, on ne peut pas. La raison est que le nom du fichier est fusionné avec les structures de données du fichier (répertoire = {nom+descripteur}). Contrairement à UNIX, où l'espace de nom est dans une structure de répertoire qui est distinct du des structures de données des fichiers (répertoire = {nom+i-noeud}).